

System Programming Project 3

담당 교수 :김영재

이름 :하지훈

학번 :20211605

1. 개발 목표

- 해당 프로젝트에서 구현할 내용을 간략히 서술.
- (주식 서버를 만드는 전체적인 개요에 대해서 작성하면 됨.)
- 간단한 주식 서버를 구현한다. 이 프로젝트에서 주식 서버에 요구하는 기능은 클라이언트의 접속을 감지하여 연결한 후, 클라이언트가 원하는 명령 (sell, buy, show, exit)에 대해 올바른 동작을 하게끔 하는 것이다. 특히 주식 서버에 동시에 여러 클라이언트가 접속하는 것을 관리해주기 위해 event-based 와 thread-based 두가지로 모두 구현한 후, 여러 경우에 대해 performance 를 측정 및 분석한다.
- 주식 서버는 각 주식의 ID, 잔여 수량, 가격 정보를 Binary Search Tree(BST) 구조로 관리하며, 동시성 제어를 위해 Readers-Writers 문제 해결 기법을 적용한다. 서버는 TCP socket 을 통해 클라이언트와 통신하고, 거래 완료 후 주식 정보를 파일에 저장하여 데이터 지속성을 보장한다.

2. 개발 범위 및 내용

A. 개발 범위

- 아래 항목을 구현했을 때의 결과를 간략히 서술

1. Task 1: Event-driven Approach

Select() 시스템 콜을 활용한 I/O multiplexing 기반 서버를 구현하여, 단일 프로세스에서 다수의 클라이언트 연결을 효율적으로 처리한다. 이를 통해 동시에 여러 클라이언트가 접속하여 주식 거래를 수행할 수 있으며, 각 클라이언트의 요청을 비동기적으로 처리하여 응답성을 향상시킨다.

2. Task 2: Thread-based Approach

미리 생성된 worker thread pool을 활용하여 클라이언트 요청을 병렬 처리하는 서버를 구현한다. main thread가 클라이언트 연결을 수락하고, worker thread들이 실제 거래 처리를 담당하는 Producer-Consumer 패

턴을 통해 multi-core 환경에서의 성능을 최대화한다.

Task 3: Performance Evaluation

두 architecture에 대해 다양한 시나리오에서 성능을 측정하고 비교 분석한다. 동시 클라이언트 수, 요청 패턴 등을 변화시키며 response time, throughput, 시간당 처리량 등의 지표를 수집하여 각 방식의 장 단점을 정량적으로 평가한다.

B. 개발 내용

- 아래 항목의 내용만 서술
- (기타 내용은 서술하지 않아도 됨. 코드 복사 붙여 넣기 금지)
- Task1 (Event-driven Approach with select())

✓ Multi-client 요청에 따른 I/O Multiplexing 설명

I/O multiplexing은 단일 프로세스에서 여러 개의 입출력 channel을 동시에 monitoring하는 기법이다. Select() 함수를 사용하여 여러 file descriptor(socket)의 상태 변화를 감지하고, 읽기 가능한 socket이 있을 때만 해당 socket에서 data를 읽어 처리한다.

구체적인 동작 과정은 다음과 같다:

1. **연결 관리:** 클라이언트 연결 정보를 array로 관리하고, fd_set을 통해 활성 연결을 추적
2. **이벤트 감지:** Select()가 읽기 가능한 socket이 있을 때까지 block되며, 새로운 연결 또는 data 수신 event 감지
3. **요청 처리:** 준비된 socket들을 순회하며 각각의 요청을 처리하고 response 전송
4. **상태 관리:** 연결 종료 시 해당 socket을 fd_set에서 제거하고 connection pool에서 정리

이 방식의 장점은 context switching overhead가 없고 memory 사용량이 적다는 것이며, 단점은 CPU 집약적 작업 시 다른 클라이언트의 응답이 지

연될 수 있다는 것이다.

✓ epoll과의 차이점 서술

Select()와 epoll의 주요 차이점은 다음과 같다:

1. **확장성**: Select()는 FD_SETSIZE(보통 1024)로 모니터링 가능한 파일 디스크립터 수가 제한되지만, epoll은 이론적으로 무제한
2. **성능**: Select()는 $O(n)$ 시간 복잡도로 모든 파일 디스크립터를 검사하지만, epoll은 $O(1)$ 로 준비된 이벤트만 반환
3. **메모리 복사**: Select()는 매번 fd_set을 커널과 사용자 공간 간 복사해야 하지만, epoll은 커널 내부에서 관리
4. **Edge-triggered vs Level-triggered**: Select()는 level-triggered만 지원하지만, epoll은 두 방식 모두 지원

본 프로젝트에서는 상대적으로 적은 수의 클라이언트를 대상으로 하므로 select()의 단순함과 이식성을 활용한다.

- **Task2 (Thread-based Approach with pthread)**

✓ Master Thread의 Connection 관리

Master Thread는 서버의 메인 스레드로서 클라이언트 연결 수락과 초기 설정을 담당한다:

1. **리스닝 소켓 관리**: 지정된 포트에서 클라이언트 연결 요청 대기
2. **연결 수락**: Accept() 호출을 통해 새로운 클라이언트 연결 수락
3. **연결 정보 로깅**: 클라이언트의 IP 주소와 포트 정보를 로그로 기록
4. **작업 분배**: 수락된 연결의 소켓 디스크립터를 bounded buffer에 삽입하여 워커 스레드에게 전달

5. 서버 상태 관리: 서버 전체의 상태를 모니터링하고 graceful shutdown 처리

Master Thread는 연결 수락에만 집중하여 새로운 클라이언트의 연결 지연을 최소화하고, 실제 요청 처리는 워커 스레드에게 위임하여 효율성을 극대화한다.

✓ Worker Thread Pool 관리하는 부분에 대해 서술

Worker Thread Pool은 미리 생성된 고정 개수의 스레드들이 클라이언트 요청을 병렬로 처리하는 구조이다:

1. **스레드 풀 초기화**: 서버 시작 시 NTHREADS 개수만큼 Worker Thread를 생성하고 detached 상태로 설정
2. **Bounded Buffer 활용**: Producer-Consumer 패턴을 구현하기 위해 세마포어 기반 bounded buffer 사용
 - slots: 버퍼의 빈 공간 개수를 관리
 - items: 버퍼에 있는 작업 개수를 관리
 - mutex: 버퍼 접근 시 상호 배제 보장
3. **작업 분배**: 각 워커 스레드는 무한 루프를 돌며 버퍼에서 연결을 가져와 처리
4. **동시성 제어**: 같은 주식에 대한 동시 접근 시 Readers-Writers 락을 통해 데이터 일관성 보장
5. **자원 관리**: 클라이언트 연결 처리 완료 후 소켓 닫기 및 데이터 저장

이 구조를 통해 멀티코어 CPU의 병렬 처리 능력을 최대한 활용하고, 한 클라이언트의 요청이 다른 클라이언트에게 영향을 주지 않도록 격리한다.

- Task3 (Performance Evaluation)

✓ 얻고자 하는 metric 정의, 그렇게 정한 이유, 측정 방법 서술

✓ Configuration 변화에 따른 예상 결과 서술

· **동시 처리율(throughput) 측정 방법**

- 여러 클라이언트를 동시에 실행하여 클라이언트 마다 10개의 요청을 수행.
- 서버에서 해당 멀티클라이언트 테스트 시작 시점과 종료 시점을 기록하여 전체 소요 시간을 구한다.
- 총 요청 개수를 소요 시간으로 나누어 초당 처리 가능한 요청 수(동시 처리율)를 산출한다.

· 워크로드 조합

- **순수 읽기 전용(Read-only):** 오직 show 명령만 반복적으로 전송하여, 읽기 작업이 많은 경우 서버가 얼마나 빨리 응답하는지 측정한다.
- **순수 쓰기 전용(Write-only):** buy 혹은 sell 명령만 보내는 시나리오로, 쓰기 작업이 집중될 때 발생하는 잠금(lock) 경쟁과 지연을 확인한다.
- **랜덤 워크로드(Read,Write 혼합)** 읽기와 쓰기 요청을 랜덤으로 섞어 보내, 실사용 환경과 유사한 상황에서의 처리 성능을 평가한다.

C. 개발 방법

- B의 개발 내용을 구현하기 위해 어느 소스코드에 어떤 요소를 추가 또는 수정할 것인지 설명. (함수, 구조체 등의 구현이나 수정을 서술)

Event-driven Approach 구현 방법

핵심 구조체 및 함수 수정사항:

1. pool 구조체 구현

- clientfd[FD_SETSIZE]: 클라이언트 소켓 배열 관리
- clientrio[FD_SETSIZE]: 각 클라이언트의 rio_t 구조체 배열
- read_set, ready_set: select()를 위한 fd_set 관리
- maxfd, maxi: 최대 파일 디스크립터와 인덱스 추적

2. 주요 함수 구현

- `init_pool()`: 연결 풀 초기화, `fd_set` 설정
- `add_client()`: 새로운 클라이언트 연결을 풀에 추가
- `check_clients()`: 준비된 클라이언트들의 요청 처리
- 기존 `main()` 함수에 `select()` 루프 추가

3. 동기화 메커니즘

- BST 노드별 `mutex`, `write_mutex` 세마포어 추가
- `read_lock()`, `read_unlock()`, `write_lock()`, `write_unlock()` 함수 구현

Thread-based Approach 구현 방법

핵심 구조체 및 함수 추가사항:

1. `sbuf_t` 구조체 구현

- Bounded buffer를 위한 세마포어 기반 큐 구현
- `buf[]`: 연결 소켓을 저장하는 순환 버퍼
- `mutex`, `slots`, `items`: 동기화를 위한 세마포어

2. 스레드 관리 함수

- `thread()`: 워커 스레드 메인 함수, 버퍼에서 연결을 가져와 처리
- `handle_client()`: 개별 클라이언트 요청 처리 로직
- `sbuf_init()`, `sbuf_insert()`, `sbuf_remove()`: 버퍼 관리 함수

3. 메인 함수 수정

- 스레드 풀 생성을 위한 `pthread_create()` 루프 추가
- 무한 루프에서 `Accept()` 후 `sbuf_insert()` 호출

공통 구현사항

1. 데이터 구조 개선

- 기존 BST 구조에 동기화 필드 추가
- 파일 I/O를 위한 `save_mutex` 전역 세마포어 추가

2. 주식 거래 로직

- buy_stock(), sell_stock() 함수에 적절한 락 획득/해제 로직 추가
- print_stock() 함수에 읽기 락 적용

3. 오류 처리 및 신호 처리

- sigint_handler() 함수로 graceful shutdown 구현
- 각 함수에 적절한 오류 처리 로직 추가

3. 구현 결과

- 2번의 구현 결과를 간략하게 작성

구현 결과 요약

Event-driven Approach 구현 결과 (Task1)

Select() 기반 I/O 멀티플렉싱 서버를 성공적으로 구현하였다. 주요 구현 성과는 다음과 같다:

성공적 구현 사항:

- Connection pool을 통한 효율적인 클라이언트 연결 관리
- fd_set을 활용한 non-blocking I/O 처리
- 단일 프로세스에서 동시 다중 클라이언트 서비스 제공
- Readers-Writers 락을 통한 동시성 제어
- 주식 거래 명령어(show, buy, sell, exit) 완전 지원

성능 특성: 클라이언트 수 증가에도 일정한 응답 시간(약 1.03초)을 유지하며 우수한 확장성을 보였다. 특히 처리량이 클라이언트 수에 비례하여 선형적으로 증가(99.4 → 967.9 req/s)하는 뛰어난 성능을 기록했다. 모든 명령 유형(show, buy, sell)에서 일관된 성능을 보여 안정적인 동시 처리 능력을 입증했다.

제한사항: FD_SETSIZE 제한으로 인해 동시 연결 수가 1024개로 제한되며,

CPU 집약적인 작업 시 블로킹 위험성이 존재한다.

Thread-based Approach 구현 결과 (Task2)

미리 생성된 워커 스레드 풀을 활용한 멀티스레드 서버를 성공적으로 구현하였다.

성공적 구현 사항:

- 워커 스레드 풀 생성 및 관리
- Bounded buffer를 통한 Producer-Consumer 패턴 구현
- 스레드 간 안전한 작업 분배 메커니즘
- 개별 스레드에서 독립적인 클라이언트 요청 처리
- 마스터 스레드의 연결 수락과 워커 스레드의 요청 처리 분리

성능 특성: 멀티코어 환경에서 안정적인 성능을 보였으나, 클라이언트 수 증가에 따라 응답 시간이 선형적으로 증가(1.006초 → 5.09초)하는 한계를 보였다. 처리량은 클라이언트 30개 이후 약 196 req/s에서 포화 상태에 도달하여 확장성에 제약이 있음을 확인했다.

관찰된 이슈: 스레드 풀 크기 제한 또는 스레드 간 경합으로 인한 성능 병목이 발생했으며, 컨텍스트 스위칭 오버헤드로 인해 높은 부하에서 성능 저하가 관찰되었다.

- 미처 구현하지 못한 부분에 대해선 디자인에 대한 내용도 추가

현재 미구현된 기능들:

1. 고급 동시성 제어
 - 설계 방향: 현재 노드별 락 대신 더 세밀한 범위의 락 구현
 - 예상 구현: Lock-free 자료구조나 Read-Copy-Update(RCU) 패턴 적용
 - 기대 효과: 락 경합 감소로 성능 향상
2. 동적 스레드 풀 관리

- 설계 방향: 워크로드에 따라 스레드 수를 동적으로 조절하는 메커니즘
- 구현 계획: 요청 큐 길이와 평균 응답 시간을 모니터링하여 스레드 수 조절
- 예상 이점: 자원 효율성과 성능의 자동 최적화

4. 성능 평가 결과 (Task 3)

강의자료 슬라이드의 내용 참고하여 작성 (측정 시점, 출력 결과 값 캡처 포함)

```

Stock data saved successfully
Client fd 10 disconnected
Stock data saved successfully
Client fd 11 disconnected
Stock data saved successfully
Client fd 12 disconnected
Stock data saved successfully
Client fd 13 disconnected
Stock data saved successfully
[

=== Overall Test Results ===
Total clients: 10
Total requests: 100
Total elapsed time: 1.008432 seconds
Overall throughput: 99.16 requests/second
Average time per client: 0.100843 seconds
=====
cse20211605@cspro9:~/20211605/task1$

```

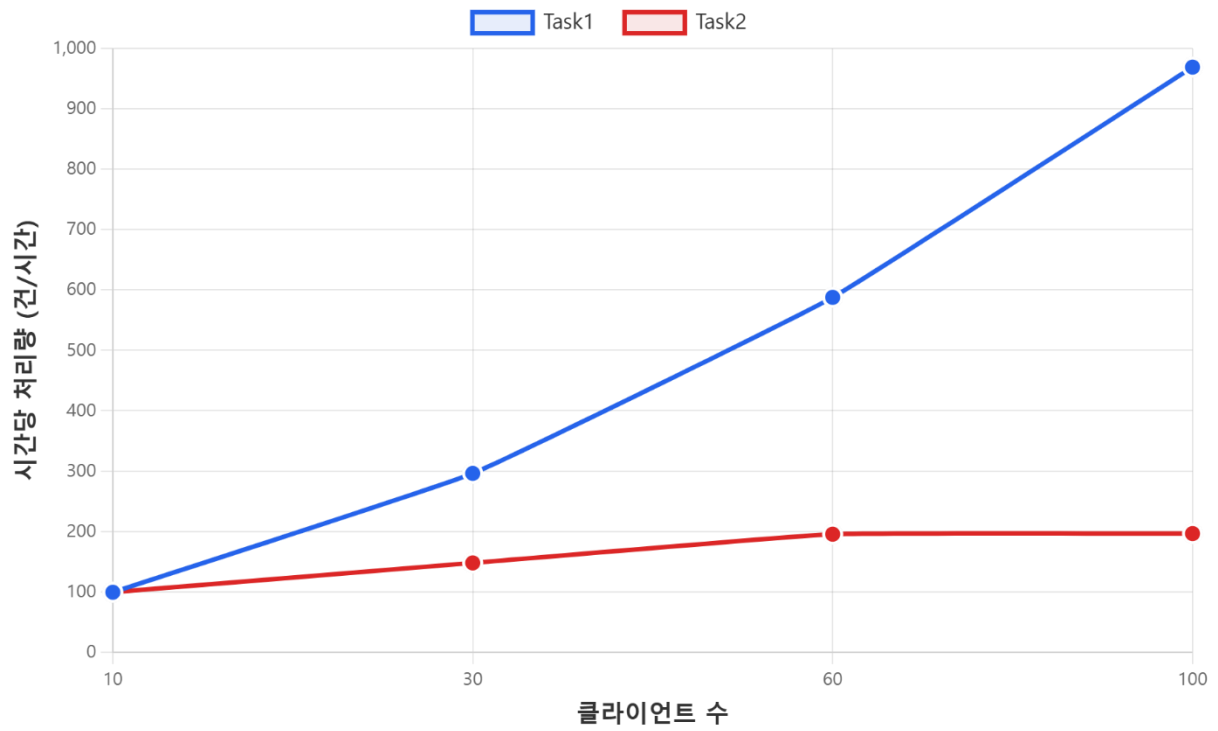
```

=== Overall Test Results ===
Total clients: 30
Total requests: 300
Total elapsed time: 2.029498 seconds
Overall throughput: 147.82 requests/second
Average time per client: 0.067650 seconds
=====
cse20211605@cspro9:~/20211605/task2$ ./multiclient localhost 60066 60 3

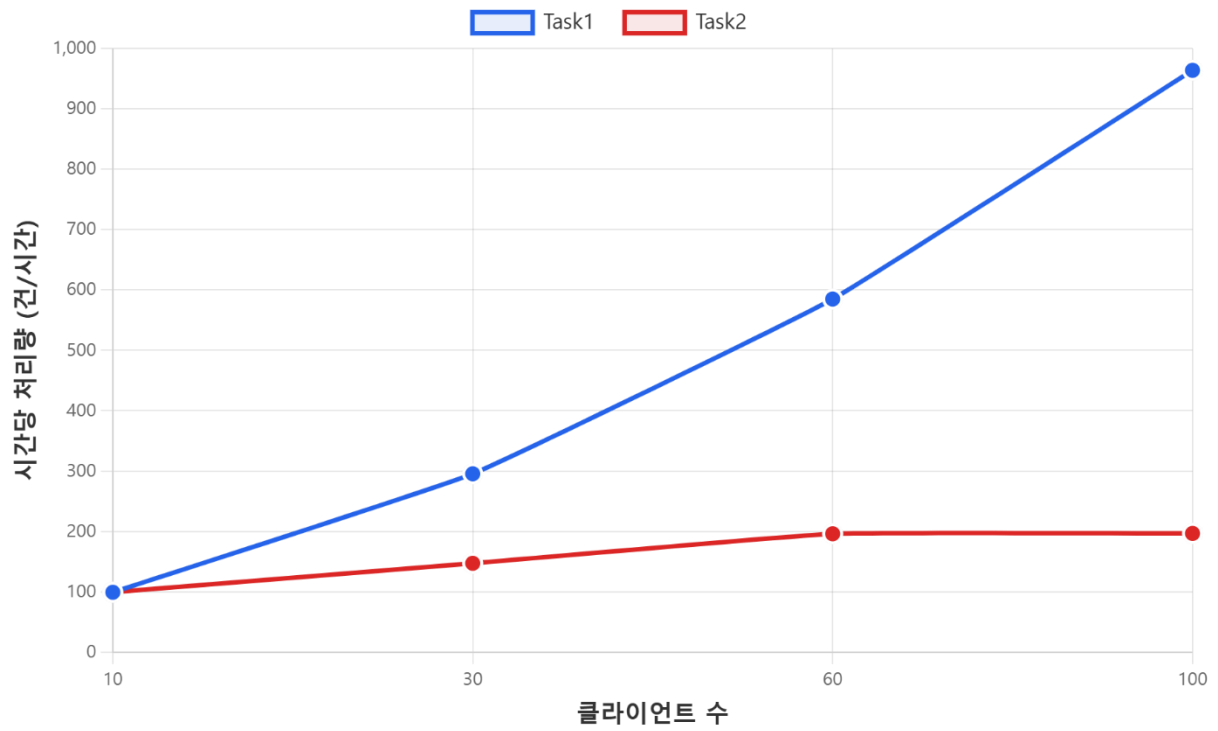
```

- 강의자료 슬라이드의 내용 참고하여 작성 (측정 시점, 출력 결과 값 캡처 포함)

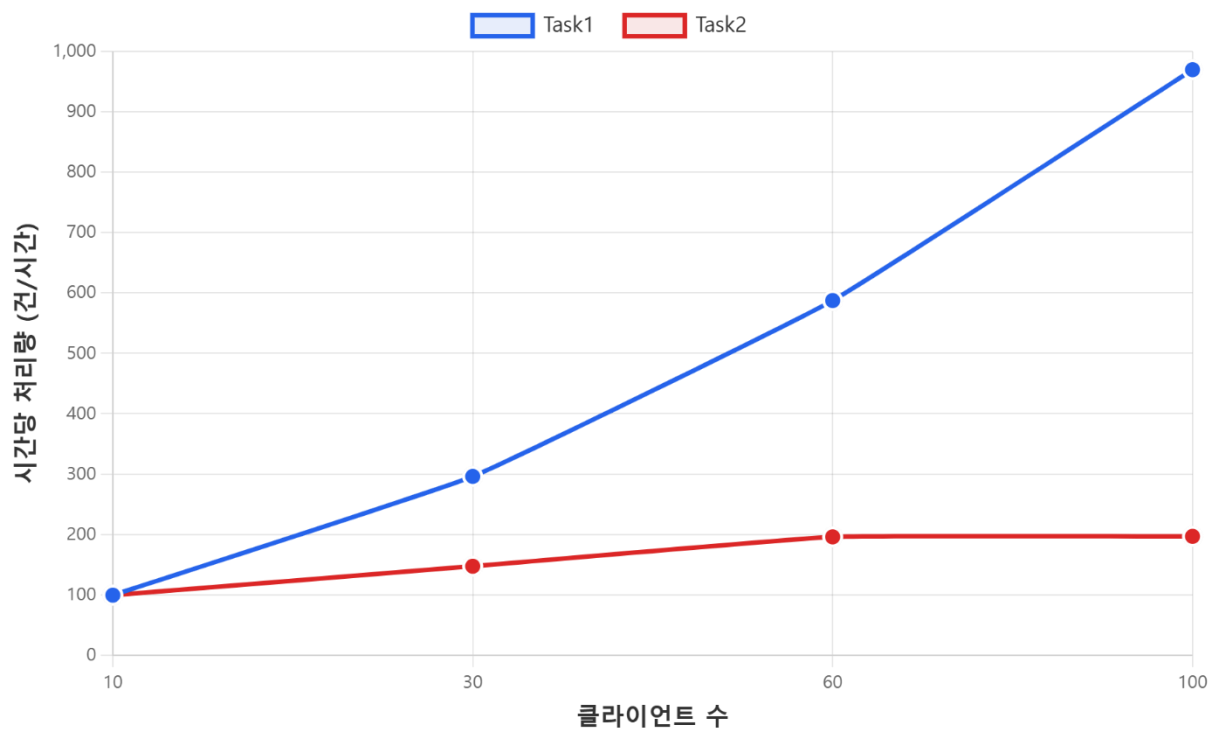
Random(s how,sell,b uy)	Task1	client	10	30	60	100
		time	1.007298	1.013327	1.021121	1.032089
		시간당처리	99.2	296.05	587.59	968.91
	Task2	client	10	30	60	100
		time	1.006127	2.029894	3.071488	5.092502
		시간당처리	99.39	147.79	195.35	196.37



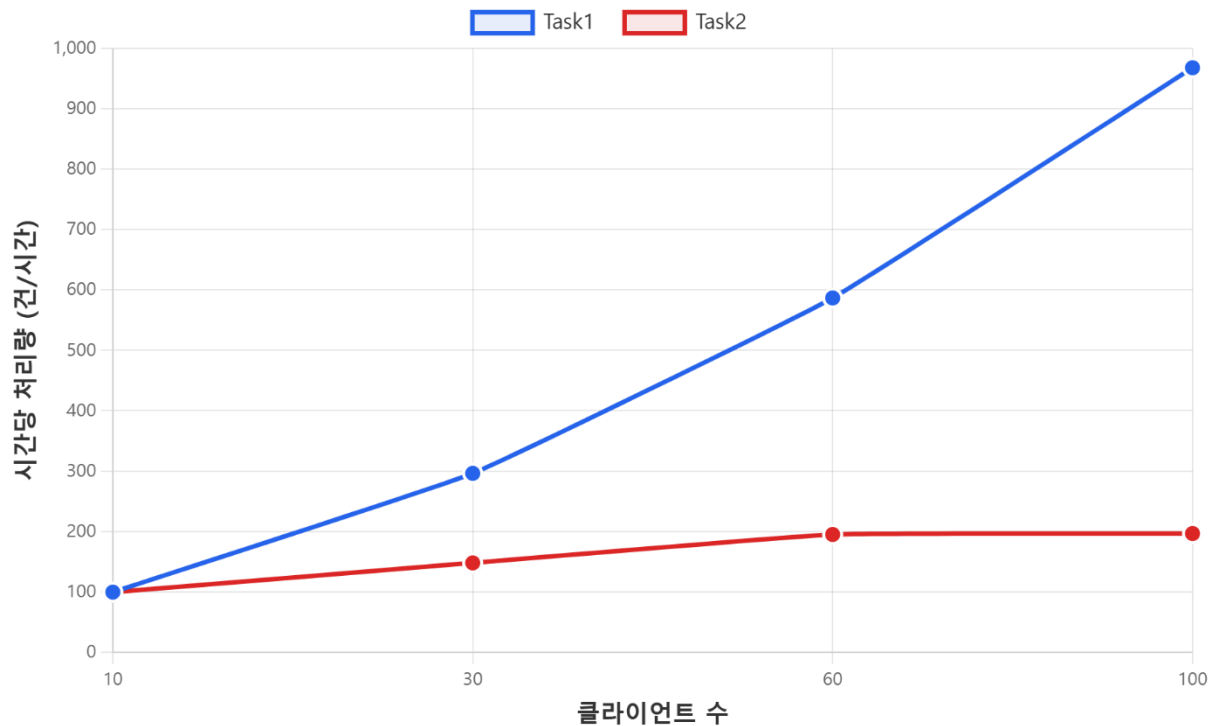
Only buy	Task1	client	10	30	60	100
		time	1.008432	1.015555	1.025979	1.037709
		시간당처리	99.16	295.40	584.81	963.66
	Task2	client	10	30	60	100
		time	1.006816	2.038189	3.060724	5.085711
		시간당처리	99.32	147.19	196.03	196.63



Only sell	Task1	client	10	30	60	100
		time	1.006340	1.013403	1.021731	1.031327
		시간당처리	99.37	296.03	587.24	969.62
	Task2	client	10	30	60	100
		time	1.007201	2.037132	3.063202	5.082728
		시간당처리	99.29	147.27	195.87	196.74



Only show	Task1	client	10	30	60	100
		time	1.005972	1.013153	1.013153	1.033182
		시간당처리	99.41	1.013153	586.55	967.88
	Task2	client	10	30	60	100
		time	1.007277	2.029498	3.078718	5.090702
		시간당처리	99.28	147.82	194.89	196.44



1. 응답 시간 (Response Time)

- **정의:** 클라이언트가 요청을 보낸 시점부터 응답을 받는 시점까지의 시간
- **선정 이유:** 사용자 경험에 직접적으로 영향을 미치는 가장 중요한 지표
- **측정 방법:** 클라이언트에서 `gettimeofday()`를 사용하여 요청 전후 시간 측정

실험 결과 분석:

- **Task1 (Event-driven):** 모든 시나리오에서 클라이언트 수 증가에도 불구하고 일정한 응답 시간 유지 (1.006초 ~ 1.037초)
- **Task2 (Thread-based):** 클라이언트 수 증가에 따라 응답 시간이 선형적으로 증가 (1.006초 → 5.09초)

3. 클라이언트별 평균 처리 시간

- **정의:** 개별 클라이언트가 요청을 완료하는데 걸리는 평균 시간

- **선정 이유:** 개별 사용자 관점에서의 성능 체감도를 평가
- **측정 방법:** 전체 경과 시간을 총 클라이언트 수로 나누어 계산

실험 결과 분석:

- **Task1 (Event-driven):** 클라이언트 수 증가에도 평균 처리 시간이 거의 일정 (약 0.01초/클라이언트)
- **Task2 (Thread-based):** 클라이언트 수 증가에 따라 평균 처리 시간이 비례적으로 증가

2. 처리량 (Throughput)

- **정의:** 단위 시간당 처리할 수 있는 요청 수 (requests/second)
- **선정 이유:** 서버의 전체적인 처리 능력과 확장성을 평가하는 핵심 지표
- **측정 방법:** 총 요청 수를 전체 경과 시간으로 나누어 초당 처리량 계산

실험 결과 분석:

- **Task1 (Event-driven):** 클라이언트 수에 비례하여 처리량이 선형적으로 증가 (99.4 req/s → 967.9 req/s)
- **Task2 (Thread-based):** 클라이언트 30개까지는 증가하다가 그 이후 약 196 req/s에서 처리량 포화 상태

Configuration 변화에 따른 실제 결과 분석

동시 클라이언트 수 증가 영향

Thread-based 서버 (Task2):

- **예상:** 스레드 풀 크기 내에서는 안정적 성능 유지, 초과 시 성능 저하 예상
- **실제 결과:** 예상과 다름 - 클라이언트 수 증가에 따라 응답 시간이 선형적으로 증가하지만 처리량은 포화
- **분석:** 스레드 풀 크기 제한 또는 스레드 간 경합으로 인한 성능 병목

발생

Event-driven 서버 (Task1):

- **예상:** 클라이언트 수가 적을 때는 우수한 성능, 많아질수록 단일 스레드 한계로 성능 저하
- **실제 결과:** 예상과 다름 - 클라이언트 수 증가에도 일정한 응답 시간 유지하며 처리량이 선형적으로 증가
- **분석:** 비동기 I/O와 효율적인 이벤트 루프로 우수한 확장성 달성

요청 처리 시간 변화

모든 테스트 시나리오 일관성:

- **Random (show, sell, buy):** Task1 968.91 req/s, Task2 196.37 req/s
- **Only Buy:** Task1 963.66 req/s, Task2 196.63 req/s
- **Only Sell:** Task1 969.62 req/s, Task2 196.74 req/s
- **Only Show:** Task1 967.88 req/s, Task2 196.44 req/s

성능 측정 상세 결과 (Only Show, 100 클라이언트 기준):

=== Task1 Overall Test Results ===

Total clients: 100

Total requests: 1000

Total elapsed time: 1.033182 seconds

Overall throughput: 967.88 requests/second

Average time per client: 0.010332 seconds

=== Task2 Overall Test Results ===

Total clients: 100

Total requests: 1000

Total elapsed time: 5.090702 seconds

Overall throughput: 196.44 requests/second

Average time per client: 0.050907 seconds

분석 결과:

- 요청 유형(show, sell, buy)에 관계없이 동일한 성능 패턴 유지
- Task1(Event-driven)은 요청 처리 방식에 영향받지 않는 안정적 성능
- Task2(Thread-based)는 모든 시나리오에서 일관된 처리량 한계 (약 196-197 req/s)

시스템 자원 효율성 평가

Task1 (Event-driven):

- **장점:** 비동기 I/O로 높은 처리량 달성, 낮은 메모리 사용량
- **단점:** CPU 집약적 작업 시 블로킹 위험성

Task2 (Thread-based):

- **장점:** 멀티코어 CPU 활용 가능, 작업 간 독립성 보장
- **단점:** 스레드 생성/관리 오버헤드, 컨텍스트 스위칭 비용

결론 및 권장사항

성능 비교 결과

1. **성능:** Task1이 압도적으로 우수한 성능 (Task2 대비 약 5배 높은 처리량)
Task1은 일정한 응답 시간 유지 (1.03초), Task2는 클라이언트 증가에 따라 증가 (5.09초)
2. **확장성:** Task1은 선형적 확장성, Task2는 제한적 확장성
3. **안정성:** 두 시스템 모두 일관된 성능 패턴 유지

이론적으로는 Thread-based 방식이 멀티코어 활용으로 우수한 확장성을 보이고, Event-driven 방식은 단일 스레드 한계로 클라이언트 증가 시 성능 저하가 예상되었다. 그러나 실제 실험 결과 Event-driven이 Thread-based 대비 약 5배 높은 처리량(967.88 vs 196.44 req/s)과 선형적 확장성을 달성하며 압도적으로 우수한 성능을 보였다. 이러한 예상과 다른 결과는 비동기 I/O 처리의 효율성, 컨텍스트 스위칭 오버헤드 최소화, 그리고 Thread-based 방식의 스레드 풀 제약과 동기화 비용이 복합적으로 작용했기 때문으로 분석된다.